Universidad de Lima

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas



IDENTIFICACIÓN DE ATAQUES DDOS TIPO UDP FLOOD HACIA DISPOSITIVOS IOT DOMÉSTICOS MEDIANTE ALGORITMOS DE MACHINE LEARNING Y DEEP LEARNING E HÍBRIDOS

Trabajo de investigación para optar el Título Profesional de Ingeniero de Sistemas

Jose Alonso Matos Carpio

Código 20223709

Giomar Fidel Castillo Ruiz

Código 20213384

Asesor

Jose Raúl Diaz Parra

Lima – Perú

Septiembre 2026

1. INTRODUCCIÓN

El Internet de las Cosas (IoT) es una red de dispositivos electrónicos equipados con tecnologías integradas que les permiten interactuar entre sí y con el entorno externo (Adamova et al., 2024, p. 427). En el contexto actual, estas redes presentan un crecimiento acelerado y se pronostica que para el año 2030 representen una inversión tecnológica mayor a 7.94 billones de dólares a nivel mundial, con expansión en múltiples sectores como salud, educación e industria (Centro Nacional de Planeamiento Estratégico [CEPLAN], 2024). Sin embargo, este crecimiento exponencial trae consigo una mayor superficie de ataque para los ciberdelincuentes, lo que se evidencia en casos recientes como el compromiso de más de 86,000 dispositivos IoT por parte de la botnet Eleven11, utilizada para lanzar ataques DDoS a gran escala (Cybersecurity Dive, 2023). Estos eventos demuestran que los dispositivos domésticos, como cámaras, routers o asistentes virtuales, pueden convertirse fácilmente en puntos de entrada y propagación de ciberataques, poniendo en riesgo tanto la infraestructura tecnológica como la privacidad de los usuarios.

Esta problemática es respaldada por Khraisat y Alazab (2021), quienes señalan que los dispositivos IoT se vuelven cada vez más prevalentes y tienen acceso a información sensible, como datos personales, lo que incrementa los riesgos relacionados con la seguridad de la información. Asimismo, los sistemas tradicionales de detección de intrusiones (IDS), diseñados para redes convencionales, presentan dificultades cuando se aplican a redes IoT heterogéneas. La falta de flexibilidad para adaptarse a la diversidad de dispositivos y protocolos reduce su eficacia. Además, los ataques avanzados, incluidos los ataques de día cero, superan fácilmente estos sistemas, ya que dependen de patrones conocidos y presentan limitaciones para procesar grandes volúmenes de datos, lo que genera problemas de rendimiento y escalabilidad (Kikissagbe & Adda, 2024, p. 1).

El impacto de estas vulnerabilidades no solo afecta a las empresas y organizaciones, sino también a los usuarios domésticos(Adamova et al., 2024) . En este contexto, surge la necesidad de proteger la seguridad de la información y salvaguardar los datos privados. Las redes IoT se encuentran entre las más expuestas a sufrir ciberataques, como DoS, DDoS, malware, entre otros, lo que incrementa la preocupación por la seguridad en estos entornos conectados(Khraisat & Alazab, 2021) .

Debido a esta problemática, la detección de ataques de Denegación de Servicio Distribuido (DDoS) en redes IoT se ha convertido en un área de investigación relevante en la comunidad

académica (Ahmed et al., 2024; Khraisat & Alazab, 2021). En la literatura reciente se han explorado enfoques basados en Machine Learning, como Random Forest y otros algoritmos de clasificación (Al Farsi et al., 2024; Ebady Manaa et al., 2024; Srivastava et al., 2024; Verma & Ranga, 2022); Deep Learning, mediante redes neuronales convolucionales (CNN) y otras arquitecturas profundas (Ebrahimi et al., 2025; Ferrag et al., 2022; Imtiaz et al., 2022); así como modelos híbridos que combinan diferentes técnicas de aprendizaje automático y profundo (Abualhassan et al., 2025; Alshamrani et al., 2023; Yang et al., 2025).

Sin embargo, a partir del análisis de la literatura, se identifican diversas limitaciones en estos trabajos. En primer lugar, se observa el uso de datasets que no siempre representan adecuadamente las condiciones de las redes IoT reales, debido a que algunos proceden de entornos controlados o sintéticos. Asimismo, existe el riesgo de sobreajuste (overfitting) durante el entrenamiento de los modelos, lo que puede afectar su capacidad de generalización. En segundo lugar, diversos estudios priorizan las métricas de precisión y clasificación, dejando en segundo plano otros aspectos importantes, como la latencia y el consumo de recursos computacionales. Finalmente, los modelos más complejos pueden presentar mayores requerimientos de procesamiento y memoria, lo que dificulta su aplicación en entornos IoT cuyos dispositivos poseen recursos limitados y requieren respuestas oportunas.

Ante estas limitaciones, surge la necesidad de continuar investigando soluciones basadas en técnicas de Machine Learning y Deep Learning que permitan fortalecer la detección de ataques DDoS en redes IoT domésticas. Por ello, la principal contribución de la presente investigación consiste en desarrollar y evaluar un prototipo de sistema de detección de intrusiones (IDS) orientado a la identificación de ataques DDoS tipo UDP Flood en redes IoT domésticas simuladas, mediante la evaluación comparativa de Machine Learning, Deep Learning e híbridos en términos de precisión, eficiencia y tiempo de respuesta. En este contexto surge la pregunta de investigación: ¿Qué diferencias presentan los modelos Machine Learning, Deep Learning e híbridos en términos de la precisión, la eficiencia y el tiempo de respuesta para la detección de ataques DDoS tipo UDP Flood en redes IoT domésticas simuladas?

Para responder a esta pregunta, la investigación adopta un enfoque cuantitativo y experimental estructurado en cinco etapas. La primera etapa se selecciona los datasets TON_IOT y se diseña el entorno IoT experimental. La segunda etapa consiste en la construcción y etiquetado del dataset experimental, pues se genera captura del trafico normal y UDP Flood para este dataset.

La tercera etapa presenta el preprocesamiento y selección de características relevantes. La cuarta etapa sería el entrenamiento y optimización de los modelos mediante hiperparametros. Finalmente, la quinta etapa corresponde a la implementación y validación donde se evalúa el prototipo IDS.

El resto del artículo se organiza de la siguiente manera. En la Sección 2 se presentan los trabajos relacionados, en la Sección 3 se desarrolla el marco teórico. Posteriormente, en la Sección 4 se describe la metodología empleada, en la Sección 5 se presenta la experimentación realizada en el entorno IoT doméstico simulado y en la Sección 6 se exponen los resultados obtenidos. Finalmente, en las Secciones 7 y 8 se presentan las conclusiones del estudio y las líneas de trabajos futuros, respectivamente.

2. TRABAJOS RELACIONADOS

Tabla 2.1

Estado del arte de detección de ataques DDoS en IoT

Autor(es)	Enfoque	Algoritmos	Ataque	Dataset	Accuracy	Aportes clave	Limitaciones
Ebady Manaa et al. (2024)	Machine Learning	Random Forest	UDP Flood, DDoS	Edge-IIoTSet	98.7%	Alta precisión en detección DDoS y bajo tiempo de respuesta	Dependencia de entornos controlados
Al Farsi et al. (2024)	Machine Learning	Random Forest	SYN Flood, UDP Flood	Edge-IIoTSet	98%	Buen equilibrio entre precisión y eficiencia computacional	Limitada validación en escenarios reales

Ebrahimi et al. (2025)	Deep Learning	CNN + Explainable AI	DDoS IoT	IoT Dataset	98%	Mejora de interpretabilidad y precisión del modelo	Riesgo de overfitting
Yang et al. (2025)	Modelo híbrido	CNN + Random Forest	SYN Flood, UDP Flood	DDoS IoT Dataset	99.2%	Mejora en precisión y generalización	Mayor complejidad computacional
Srivastava et al. (2024)	Machine Learning	Random Forest	DDoS TCP/UDP	IoT DDoS Dataset	>96%	Comparación de algoritmos ML para detección DDos	Selección de características poco optimizada
Verma & Ranga(2022)	Machine Learning	SVM, Decision Tree, RF	Botnet DDoS	Bot-IoT	95%	Identificación eficiente de tráfico malicioso en Iot	Problemas de escalabilidad
Imtiaz et al. (2022)	Deep Learning	Deep Neural Network (DNN)	SYN Flood, UDP Flood	IoT Anomaly Dataset	99%	Detección efectiva de anomalías complejas	Alto costo computacional
Ferrag et al. (2022)	Deep Learning	CNN, RNN, LSTM	Múltiples ataques DDoS	Múltiples datasets IoT	>98%	Comparación integral de técnicas DL para ciberseguridad IoT	Alto consumo de recursos
Saba et al. (2022)	ML y DL	Modelos ML/DL combinados	DDoS y Botnet	IoT Security Datasets	95% – 99%	Adaptabilidad a múltiples tipos de ataques	Complejidad de implementación
Abualhassan et al. (2025)	Modelo híbrido	ML + DL híbrido	DDoS IoT	IoT Environment Dataset	98.5%	Balance entre rendimiento y eficiencia	Validación limitada en tiempo real

Alshamrani et al. (2023)	Modelo híbrido	Hybrid Deep Learning	SYN Flood, UDP Flood	DDoS Dataset	99%	Mejora significativa en detección DDoS IoT	Elevado consumo de memoria y CPU

Fuente:*Elaboracion propia*

Con el fin de organizar los principales aportes de la literatura reciente, la Tabla 2.1 “Estado del arte de detección de ataques DDoS en IoT**”** sintetiza los trabajos relacionados con la detección de ataques DDoS en entornos IoT, considerando los enfoques utilizados, algoritmos aplicados, datasets empleados, métricas de desempeño, aportes y limitaciones identificada

El crecimiento acelerado del Internet de las Cosas (IoT) ha transformado los entornos domésticos inteligentes mediante la interconexión de múltiples dispositivos, generando beneficios en automatización y conectividad. Sin embargo, este avance también ha incrementado significativamente la superficie de ataque debido a las limitaciones de procesamiento, memoria y seguridad propias de estos dispositivos, lo que los convierte en objetivos frecuentes de ciberataques como Denial of Service (DoS) y Distributed Denial of Service (DDoS). En este contexto, diversos estudios destacan la necesidad de desarrollar mecanismos de detección más avanzados basados en inteligencia artificial (Adamova et al., 2024; Khraisat & Alazab, 2021).

En la literatura reciente, se han propuesto diversos enfoques para la detección de intrusiones en redes IoT, los cuales pueden agruparse en tres categorías principales: Machine Learning, Deep Learning y modelos híbridos.

En primer lugar, los enfoques basados en Machine Learning han sido ampliamente utilizados debido a su eficiencia computacional. Estudios como los de Ebady Manaa et al. (2024) y Al Farsi et al. (2024) destacan el uso de Random Forest, alcanzando precisiones superiores al 98% en datasets como Edge-IIoTSet. De manera complementaria, Srivastava et al. (2024) evidencian que Random Forest ofrece un mejor rendimiento frente a otros algoritmos como Naive Bayes en la detección de ataques DDoS. Asimismo, investigaciones recientes como las de Verma y Ranga (2022) resaltan la efectividad de los modelos de Machine Learning en la identificación de patrones de tráfico malicioso en entornos IoT.

Por otro lado, los enfoques basados en Deep Learning han demostrado una mayor capacidad para identificar patrones complejos en el tráfico de red. Imtiaz et al. (2022) proponen el uso de redes neuronales profundas para la detección de anomalías, logrando altos niveles de precisión. De manera similar, Ebrahimi et al. (2025) utilizan redes neuronales convolucionales (CNN) combinadas con técnicas de inteligencia artificial explicable, mejorando tanto el rendimiento como la interpretabilidad de los modelos. En esta misma línea, Ferrag et al. (2022) presentan un análisis comparativo de técnicas de Deep Learning aplicadas a ciberseguridad, evidenciando mejoras significativas en la detección de patrones complejos.

Asimismo, estudios como los de Saba et al. (2022) destacan los avances recientes en el uso de Machine Learning y Deep Learning en ciberseguridad para IoT, señalando su capacidad para adaptarse a diferentes tipos de ataques y escenarios.

En una línea más reciente, los modelos híbridos han surgido como una alternativa prometedora. Yang et al. (2025) proponen un modelo que integra CNN y Random Forest, logrando mejoras en la precisión y capacidad de generalización. De manera complementaria, Abualhassan et al. (2025) concluyen que la combinación de enfoques permite equilibrar rendimiento y eficiencia. En este contexto, Alshamrani et al. (2023) proponen un modelo híbrido basado en Deep Learning para la detección de ataques DDoS en redes IoT, obteniendo mejoras significativas en precisión y eficiencia.Con el fin de sintetizar y comparar los principales enfoques identificados en la literatura, se presenta la Tabla 2.1, donde se resumen los algoritmos utilizados, datasets, niveles de precisión y principales limitaciones de los estudios analizados.

Como se observa en la tabla, los enfoques basados en Deep Learning alcanzan mayores niveles de precisión, mientras que los modelos de Machine Learning presentan un mejor rendimiento en términos de eficiencia computacional. Asimismo, los modelos híbridos emergen como una alternativa prometedora al combinar las ventajas de ambos enfoques.No obstante, a partir del análisis de la literatura, se identifican diversas limitaciones. En primer lugar, se evidencia un problema recurrente de sobreajuste (overfitting) en los modelos, especialmente en aquellos basados en Deep Learning, los cuales presentan un alto rendimiento en datos de entrenamiento pero una baja capacidad de generalización en escenarios reales. Asimismo, se observa el uso de datasets poco representativos, frecuentemente generados en entornos controlados o sintéticos, lo que reduce la validez externa de los resultados.

Por otro lado, la mayoría de los estudios validan sus modelos en entornos simulados o de laboratorio, sin considerar la complejidad y variabilidad de redes IoT domésticas reales. A esto se suma el alto costo computacional de los modelos avanzados, lo cual dificulta su implementación en dispositivos con recursos limitados. Finalmente, se identifica una brecha significativa entre el rendimiento teórico y la aplicabilidad práctica, ya que pocos trabajos consideran restricciones reales como hardware limitado, consumo energético y despliegue en tiempo real.

En este contexto, se evidencia la necesidad de desarrollar modelos que logren equilibrar alta precisión, eficiencia computacional y aplicabilidad en entornos IoT domésticos. Por ello, el presente trabajo propone un enfoque híbrido basado en la combinación de Random Forest y redes neuronales convolucionales (CNN), con el objetivo de mejorar la detección de ataques DDoS sin comprometer el rendimiento en dispositivos con recursos limitados. A pesar de los avances obtenidos en la detección de ataques DDoS mediante técnicas de Machine Learning y Deep Learning, la mayoría de los trabajos revisados presenta limitaciones importantes. En muchos casos, los modelos son evaluados utilizando datasets sintéticos o generados en entornos controlados, lo que reduce la capacidad de generalización de los resultados hacia escenarios IoT domésticos reales. Asimismo, varios estudios priorizan únicamente métricas de precisión, dejando de lado aspectos relevantes como el consumo de recursos, la latencia y la viabilidad de implementación en dispositivos con capacidades limitadas.

De igual manera, los enfoques basados en Deep Learning suelen requerir altos niveles de procesamiento y entrenamiento, lo cual dificulta su despliegue en entornos IoT reales. Aunque los modelos híbridos han mostrado mejoras en rendimiento, aún existen desafíos relacionados con la complejidad de implementación, la optimización de hiperparámetros y la adaptación frente a ataques nuevos o variantes desconocidas. En consecuencia, se evidencia la necesidad de desarrollar soluciones más equilibradas, capaces de combinar precisión, eficiencia computacional y aplicabilidad práctica.

3. MARCO TEÓRICO

El presente marco teórico integra los fundamentos conceptuales, metodológicos y tecnológicos necesarios para el desarrollo de un sistema de detección de intrusiones (IDS) orientado a la identificación de ataques DDoS tipo UDP Flood en entornos IoT domésticos. Para ello, se

revisan conceptos relacionados con la seguridad en redes IoT, los ataques DDoS, los sistemas de detección de intrusiones y la aplicación de técnicas de Machine Learning y Deep Learning para la clasificación de tráfico normal y malicioso. Asimismo, se consideran estudios recientes que evidencian la necesidad de implementar mecanismos de detección más eficientes, adaptables y adecuados para redes IoT con recursos limitados**.**

3.1. Seguridad en redes IoT y sus vulnerabilidades

El Internet de las Cosas (IoT) comprende una red de dispositivos físicos interconectados que incorporan sensores, software y capacidades de comunicación para recopilar, procesar e intercambiar información a través de Internet (Adamova et al., 2024). En entornos domésticos, estos dispositivos pueden incluir cámaras inteligentes, routers, asistentes virtuales, sensores ambientales, enchufes inteligentes, focos inteligentes, cerraduras inteligentes y electrodomésticos conectados, los cuales permiten automatizar tareas, monitorear espacios y mejorar la conectividad del hogar (Khraisat & Alazab, 2021).

Sin embargo, el crecimiento acelerado de los dispositivos IoT también ha incrementado los riesgos de seguridad informática. Muchos de estos equipos cuentan con recursos limitados de procesamiento, memoria y almacenamiento, lo que dificulta la implementación de mecanismos avanzados de protección, como cifrado robusto, autenticación fuerte o actualizaciones automáticas de firmware. Esta situación incrementa la vulnerabilidad de los dispositivos frente a accesos no autorizados, explotación de credenciales débiles y ataques dirigidos contra la disponibilidad de la red (Khraisat & Alazab, 2021).

Khraisat y Alazab (2021) destacan que los sistemas tradicionales de seguridad no fueron diseñados para entornos IoT debido a la diversidad de dispositivos y la escalabilidad de estas redes. Asimismo, Adamova et al. (2024) señalan que el aumento masivo de dispositivos conectados incrementa considerablemente la superficie de ataque y la exposición a amenazas cibernéticas.

3.2. Ataques DDoS en entornos IoT

Los ataques de Denegación de Servicio Distribuido (DDoS) representan una de las principales amenazas para las redes IoT, debido a que afectan directamente la disponibilidad de los servicios y dispositivos conectados. Este tipo de ataque busca saturar un servidor, dispositivo o red mediante el envío masivo de solicitudes provenientes de múltiples equipos comprometidos, comúnmente organizados en botnets (Ahmed et al., 2024; Pakmehr et al., 2024).

En entornos IoT, los ataques DDoS son especialmente peligrosos debido a que muchos dispositivos poseen configuraciones de seguridad débiles, credenciales vulnerables y mecanismos limitados de actualización o autenticación. Como consecuencia, los atacantes pueden comprometer una gran cantidad de dispositivos y utilizarlos para generar tráfico malicioso de gran volumen hacia un objetivo específico (Khraisat & Alazab, 2021; Pakmehr et al., 2024).

Uno de los ataques más comunes es el UDP Flood, el cual consiste en el envío masivo de paquetes UDP hacia uno o varios puertos de un dispositivo o servidor objetivo, con el propósito de consumir recursos de procesamiento y ancho de banda. Este tipo de ataque puede generar congestión en servidores o dispositivos IoT, afectar su operación normal y limitar su capacidad para procesar tráfico legítimo o ejecutar mecanismos de detección de ataques (Gelenbe & Nasereddin, 2023).

Ahmed et al. (2024) señalan que los ataques DDoS en IoT han aumentado considerablemente debido a la facilidad con la que los dispositivos pueden ser comprometidos. Del mismo modo, Alahmadi et al. (2023) destacan que estos ataques afectan directamente la disponibilidad de los servicios y generan pérdidas de información y rendimiento en las redes.

3.3. Sistemas de detección de intrusiones (IDS)

Los Sistemas de Detección de Intrusiones (IDS) son herramientas de seguridad orientadas al monitoreo y análisis del tráfico de red con el objetivo de identificar actividades sospechosas, anómalas o maliciosas. En redes IoT, estos sistemas resultan esenciales para fortalecer la seguridad, debido a la diversidad de dispositivos conectados, la generación constante de tráfico y la exposición a ataques que pueden comprometer la disponibilidad, integridad o confidencialidad de los servicios (Rahman et al., 2024).

Tradicionalmente, los IDS se clasifican según su metodología de detección en sistemas basados en firmas, anomalías y especificaciones. En este trabajo se consideran principalmente los dos enfoques más utilizados: los IDS basados en firmas, que detectan ataques conocidos mediante la comparación del tráfico con patrones previamente almacenados, y los IDS basados en anomalías, que identifican desviaciones respecto al comportamiento normal de la red. Aunque los primeros presentan alta precisión frente a amenazas conocidas, poseen limitaciones para detectar ataques nuevos o variantes desconocidas, debido a que dependen de firmas previamente registradas y actualizadas; mientras que los segundos pueden detectar amenazas no conocidas, pero pueden

generar mayores tasas de falsos positivos si el comportamiento normal no se encuentra adecuadamente modelado (Bhavsar et al., 2023; Kikissagbe & Adda, 2024).

Kikissagbe y Adda (2024) señalan que los IDS tradicionales presentan dificultades en entornos IoT debido a la heterogeneidad de dispositivos, la diversidad de protocolos, la gran cantidad de tráfico generado y la limitada capacidad para adaptarse rápidamente a ataques emergentes. Estas restricciones reducen su efectividad en redes IoT domésticas, donde los dispositivos suelen operar con recursos computacionales limitados y patrones de comunicación variables. En consecuencia, los enfoques tradicionales pueden resultar insuficientes para detectar ataques nuevos o variantes desconocidas, lo que justifica el uso de técnicas basadas en Machine Learning y Deep Learning para analizar el tráfico de red y mejorar la detección de intrusiones.

3.4. Machine Learning en detección de DDoS

El Machine Learning ha sido ampliamente utilizado en sistemas de detección de intrusiones debido a su capacidad para aprender patrones a partir de datos y clasificar el tráfico de red como normal o malicioso. En este sentido, Talukder et al. (2024) señalan que los algoritmos de ML aplicados a IDS permiten identificar patrones de comportamiento normal y anomalías que pueden representar amenazas de seguridad.

Dentro de los algoritmos más utilizados destaca Random Forest, un método de aprendizaje supervisado basado en múltiples árboles de decisión. Este algoritmo utiliza técnicas de ensemble learning y bootstrap aggregation, también conocido como bagging, para mejorar la precisión del modelo y reducir el riesgo de sobreajuste (Sakr et al., 2024).

El funcionamiento de Random Forest consiste en generar múltiples árboles de decisión utilizando subconjuntos aleatorios de datos y características. Posteriormente, cada árbol realiza una predicción y el resultado final se obtiene mediante votación mayoritaria.

Por otro lado, XGBoost (Extreme Gradient Boosting) es un algoritmo de aprendizaje automático que utiliza un modelo aditivo basado en múltiples árboles de decisión para obtener una predicción final. Su funcionamiento se desarrolla de manera secuencial, incorporando nuevos árboles que buscan corregir los errores de predicción de los árboles anteriores. Durante este proceso, el algoritmo utiliza una función objetivo que combina una función de pérdida, encargada de medir los errores de predicción, y un término de regularización que controla la complejidad del modelo para reducir el sobreajuste. De esta manera, XGBoost mejora progresivamente las predicciones mediante la optimización de sus árboles de decisión (Zhang et al., 2022).

No obstante, el rendimiento de los modelos de Machine Learning depende en gran medida de la calidad de los datos, el balance de clases y la correcta selección de características relevantes del tráfico de red. En esta línea, Ngo et al. (2023) señalan que los métodos de detección de intrusiones basados en Machine Learning suelen depender de técnicas de selección o extracción de características para reducir la dimensionalidad de los datos y mejorar la eficiencia del proceso de detección.

3.5. Deep Learning en detección de DDoS

El Deep Learning ha permitido importantes avances en la detección de intrusiones debido a su capacidad para identificar relaciones complejas dentro de grandes volúmenes de datos. En el contexto de redes IoT, estos modelos se emplean para analizar tráfico de red, reconocer comportamientos anómalos y mejorar la detección de ataques como DoS y DDoS (Ferrag et al., 2022; Imtiaz et al., 2022).

Entre las arquitecturas más utilizadas destacan las Redes Neuronales Convolucionales (CNN), las cuales son capaces de extraer automáticamente características relevantes mediante operaciones de convolución y pooling. Las capas convolucionales permiten identificar patrones locales dentro de los datos, mientras que las capas de pooling reducen la dimensionalidad y optimizan el procesamiento de información. En sistemas IDS aplicados a IoT, esta capacidad resulta útil para analizar patrones del tráfico de red y diferenciar comportamientos normales de comportamientos maliciosos (Ebrahimi et al., 2025; Ferrag et al., 2022)

Una de estas redes son las redes neuronales convolucionales unidimensionales (1D-CNN) que presentan una arquitectura de aprendizaje profundo diseñada para procesar señales y datos secuenciales. Su funcionamiento se basa en la aplicación de filtros convolucionales sobre los datos de entrada, utilizando conexiones locales y pesos compartidos para extraer automáticamente características relevantes. Asimismo, incorpora capas de convolución y submuestreo (pooling), que permiten generar representaciones de las características para su posterior clasificación. Chen et al. (2022) desarrollaron un modelo basado en 1D-CNN para extraer características de señales multivariadas y aplicarlo a la detección y reconocimiento de fallos en procesos industriales. Por su parte, Assy et al. (2023) propusieron un sistema de detección de intrusiones basado en esta arquitectura, utilizando tráfico de red para identificar comportamientos anómalos. Estos estudios demuestran la aplicación de las 1D-CNN tanto en el análisis de señales como en la detección de intrusiones en redes.

En el contexto de la ciberseguridad, las CNN permiten analizar patrones complejos de tráfico de red y detectar comportamientos anómalos asociados a ataques DDoS. Imtiaz et al. (2022) proponen un modelo basado en Deep Learning que logra altos niveles de precisión en la detección de anomalías en redes IoT. De manera similar, Ebrahimi et al. (2025) destacan el uso de CNN combinadas con técnicas de inteligencia artificial explicable para mejorar tanto el rendimiento como la interpretabilidad de los modelos.

Sin embargo, los modelos de Deep Learning presentan limitaciones relacionadas con el alto consumo computacional, largos tiempos de entrenamiento y riesgo de sobreajuste cuando los datasets no son suficientemente representativos.

3.6 Modelos híbridos en IoT

El crecimiento de las amenazas cibernéticas en entornos IoT ha impulsado el desarrollo de diferentes enfoques basados en Machine Learning y Deep Learning para fortalecer los sistemas de detección de intrusiones. Diversos estudios señalan que los algoritmos de Machine Learning presentan ventajas relacionadas con su eficiencia computacional y facilidad de implementación, mientras que los modelos de Deep Learning destacan por su capacidad para identificar patrones complejos en grandes volúmenes de datos de red (Ferrag et al., 2022; Saba et al., 2022).

Sin embargo, cada enfoque presenta limitaciones particulares cuando es utilizado de forma independiente, especialmente en escenarios IoT caracterizados por recursos limitados y tráfico altamente heterogéneo.Como respuesta a estas limitaciones, investigaciones recientes han propuesto modelos híbridos que combinan técnicas de Machine Learning y Deep Learning con el objetivo de aprovechar las fortalezas de ambos enfoques. Estos modelos buscan integrar la capacidad de extracción automática de características proporcionada por las redes neuronales profundas con la eficiencia de algoritmos de clasificación tradicionales, permitiendo mejorar la precisión y capacidad de generalización de los sistemas de detección de intrusiones (Kikissagbe & Adda, 2024; Bankó et al., 2025). En este contexto, Yang et al. (2025) desarrollaron un modelo híbrido que integra Convolutional Neural Network (CNN) y Random Forest para la detección de intrusiones, obteniendo mejoras en el desempeño de clasificación frente a modelos individuales.

De manera similar, Alshamrani et al. (2023) reportan que la combinación de múltiples técnicas de aprendizaje permite incrementar la efectividad en la detección de ataques DDoS dentro de redes IoT, al aprovechar las capacidades complementarias de cada algoritmo. No obstante, los modelos híbridos también presentan desafíos relacionados con la complejidad de implementación, el ajuste de hiperparámetros y el consumo de recursos computacionales durante

las fases de entrenamiento y despliegue. Asimismo, su aplicación en dispositivos IoT con capacidades limitadas continúa representando un reto importante para la comunidad científica (Abualhassan et al., 2025; Bankó et al., 2025).

3.6. Mosquitto y MQTT en redes IoT

El protocolo MQTT es una tecnología de comunicación ampliamente utilizada en entornos de Internet de las Cosas debido a su diseño ligero y a su arquitectura basada en publicación/suscripción. En este modelo, los dispositivos IoT pueden actuar como publicadores o suscriptores de mensajes, mientras que un broker central administra la recepción y distribución de la información. Esta arquitectura resulta adecuada para dispositivos con recursos limitados, ya que permite el intercambio de mensajes de forma eficiente, reduciendo el consumo de ancho de banda y facilitando la comunicación máquina a máquina dentro de redes IoT (Bayılmış et al., 2022).

Dentro del funcionamiento de MQTT, el broker cumple un papel fundamental, ya que actúa como intermediario entre los clientes que envían información y aquellos que la reciben. Pazos (2024) señala que, en aplicaciones IoT, la comunicación puede ser administrada por un servidor denominado broker, el cual recibe los mensajes publicados por los clientes y los entrega a los suscriptores correspondientes. Asimismo, MQTT suele operar mediante tópicos, lo que permite organizar los mensajes de acuerdo con el tipo de dato, dispositivo o servicio que participa en la red.

Mosquitto es una de las implementaciones de broker MQTT de código abierto utilizadas en escenarios IoT y de edge computing. Su relevancia se debe a que permite implementar el modelo publicación/suscripción en entornos experimentales y reales, facilitando la comunicación entre dispositivos, sensores, aplicaciones y servidores. Dizdarevic et al. (2023) consideran a Mosquitto como una de las principales implementaciones de broker MQTT de código abierto evaluadas junto con EMQX, RabbitMQ, VerneMQ y HiveMQ, destacando su uso en pruebas de rendimiento con dispositivos y plataformas con recursos limitados.

En el contexto de la ciberseguridad en IoT, MQTT y sus brokers también son relevantes porque forman parte del tráfico habitual que puede analizarse en sistemas de detección de intrusiones. El tráfico generado mediante MQTT puede representar un comportamiento normal dentro de una red IoT, mientras que desviaciones significativas en volumen, frecuencia o tipo de paquetes pueden indicar comportamientos anómalos. Además, Di Paolo et al. (2023) señalan que la seguridad y confiabilidad de los componentes MQTT son aspectos importantes dentro del

ecosistema IoT, debido a la amplia adopción de este protocolo y a la exposición de los dispositivos conectados frente a posibles ataques.

Por ello, el uso de Mosquitto como broker MQTT permite representar de forma coherente la comunicación normal de una red IoT, especialmente cuando se busca construir una línea base de tráfico legítimo. Esta línea base puede compararse posteriormente con tráfico malicioso, como ataques DoS o DDoS, para entrenar y evaluar modelos de detección de intrusiones. De esta manera, Mosquitto no se entiende como una herramienta de ataque, sino como un componente de comunicación que permite simular el comportamiento normal de dispositivos IoT dentro de un entorno controlado.

3.7. Métricas de evaluación

Para evaluar el rendimiento de los modelos implementados se utilizarán métricas de clasificación ampliamente empleadas en investigaciones de ciberseguridad. Estas métricas serán aplicadas a todos los modelos evaluados: Random Forest base, Random Forest optimizado, CNN base y CNN optimizada. De esta manera, será posible comparar los resultados de forma uniforme y determinar qué configuración presenta mayor capacidad de detección de ataques UDP Flood frente al tráfico normal MQTT.

La métrica Accuracy permitirá medir el porcentaje total de predicciones correctas realizadas por el modelo, considerando tanto el tráfico normal como el tráfico malicioso generado por ataques DDoS tipo UDP Flood. Esta métrica permitirá conocer el desempeño general de los modelos durante la clasificación del tráfico analizado.

Asimismo, se emplearán Precision, Recall y F1-Score. Precision permitirá medir qué proporción de los registros clasificados como ataque realmente corresponden a tráfico malicioso. Recall permitirá evaluar qué proporción de ataques reales fueron detectados correctamente por el modelo. F1-Score permitirá obtener un balance entre Precision y Recall, especialmente útil cuando exista desbalance entre las clases del dataset.

La matriz de confusión permitirá representar los resultados de clasificación obtenidos por cada modelo, diferenciando entre predicciones correctas e incorrectas para las clases tráfico normal y UDP Flood. A partir de esta matriz se identificarán los verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN), permitiendo analizar de manera detallada los errores cometidos por los modelos. En particular, los falsos positivos representarán casos de tráfico normal clasificados incorrectamente como ataques, mientras que los falsos negativos corresponden a ataques UDP Flood que no fueron detectados.

Finalmente, se considerará la latencia o tiempo de respuesta como una medida complementaria para determinar el tiempo requerido por cada modelo para realizar la clasificación del tráfico. Esta métrica permitirá comparar no solo la capacidad de detección de los modelos, sino también su rapidez de respuesta durante el procesamiento de los datos.

En conclusión, la integración de los fundamentos teóricos abordados evidencia que la creciente vulnerabilidad y restricción de recursos computacionales en las redes IoT domésticas exige la evolución de los sistemas de detección de intrusiones tradicionales hacia modelos avanzados basados en Machine Learning y Deep Learning. Al establecer una línea base representativa de tráfico legítimo mediante el protocolo MQTT y el broker Mosquitto contrapuesta al volumen masivo y anómalo de los ataques DDoS tipo UDP Flood, se constituye el escenario experimental clave para el entrenamiento, optimización y evaluación de algoritmos como Random Forest y redes neuronales convolucionales (1D-CNN). Finalmente, la validación de estos modelos a través de métricas de rendimiento (Accuracy, Precision, Recall, F1-Score) y tiempos de respuesta (latencia) resulta indispensable para determinar la arquitectura de clasificación óptima que garantice la disponibilidad, precisión y eficiencia operativa en la protección de infraestructuras IoT domésticas.

4. METODOLOGÍA

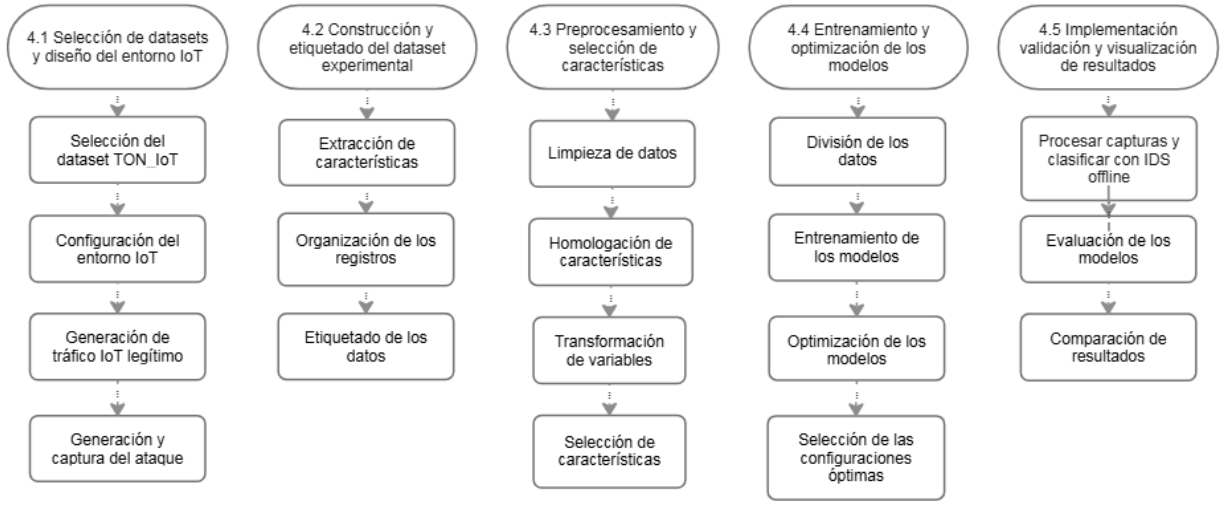
La presente investigación presenta un enfoque cuantitativo, aplicado y experimental, orientado al desarrollo de un sistema de detección de intrusiones (IDS) para clasificar tráfico de red en entorno IoT domésticos simulado. El estudio muestra el análisis comparativo entre algoritmos de Machine Learning y Deep Learning, con el fin de evaluar su capacidad de respuesta frente a ataques DDoS de tipo UDP Flood y tráfico legítimo bajo el protocolo MQTT. La contribución principal consiste en construir un dataset experimental propio en un entorno virtualizado y proponer un esquema de clasificación basado algoritmos de machine learning (Random Forest y XGBooost), deep learning (CNN 1D) y mezcla de algoritmos (CNN 1D con Random Forest y CNN 1D con Bi-LSMT), evaluando su desempeño de manera independiente frente un dataset consolidado entre un dataset público TON_IoT y el dataset experimental, para mantener variabilidad de información en la detección de tráfico malicioso frente al normal.

El desarrollo metodológico contempla cinco etapas. Primero, se selecciona el dataset TON_IoT y se configura un entorno IoT doméstico simulado con Kali Linux, Ubuntu Server, pfSense y

Docker, generando tráfico MQTT mediante Mosquitto. Luego, se captura el tráfico normal y UDP Flood mediante Wireshark y tcpdump, y se transforma los archivos PCAP a CSV, para construir el dataset experimental. Posteriormente, ambos datasets serán preprocesados y analizados de manera independiente para seleccionar las características relevantes y la consolidación de estas. En la cuarta etapa, se entrenan los modelos con los algoritmos mencionados utilizando un conjunto de entrenamiento, validación y prueba a partir del dataset consolidado. Finalmente, se evalúan y comparan los modelos mediante Accuracy, Precision, Recall, F1-Score y latencia, presentando los resultados obtenidos y su análisis respectivo..

Figura 4.1

Pipeline metodológico propuesto



Etapa 1: Selección de datasets y diseño del entorno IoT

En esta etapa se selecciona las fuentes de datos y establecer un entorno IoT doméstico simulado que permita generar tráfico legítimo y ataques UDP Flood de manera controlada y aislada.

Actividades:

1. **Selección del dataset TON_IoT:** Se selecciona y verifica que el dataset público cumpla con el propósito de esta investigación, relacionado a ataques a dispositivos a redes con dispositivos IoT en preferencia domésticos. Para ello se seleccionó el dataset público TON_IoT, identificando las características y etiquetas relacionadas con tráfico de red y ataques..
2. **Configuración del entorno IoT simulado:** Se implementa el escenario mediante máquinas virtuales, utilizando Kali Linux como máquina atacante, Ubuntu Server como servidor víctima y pfSense para la segmentación de las redes entre el atacante y la

víctima. Además del uso de contenedores Docker, para gestionar los dispositivos IoT simulados..

3. **Generación de tráfico IoT legítimo:** Se utiliza la herramienta Mosquitto como broker MQTT para representar la comunicación normal de dispositivos IoT domésticos simulados, como cámaras y un enchufe inteligente.
4. **Generación y captura del ataque:** Se genera tráfico UDP Flood desde la maquina virtual del Kali Linux hacia los dispositivos IoT simulados alojados en el Ubuntu Server, y por último se realiza la captura el tráfico neto mediante Wireshark y tcpdump para obtener los archivos PCAP.

Etapa 2: Construcción y etiquetado del dataset experimental

En esta etapa se construye el dataset experimental propio a partir de las capturas de tráfico obtenidas en el entorno IoT simulado, diferenciando el tráfico normal del tráfico correspondiente al ataque UDP Flood.

Actividades:

1. **Extracción de características:** Se convierte los archivos PCAP a formato CSV mediante el Tshark, un analizador de protocolo de red permitiendo visualizar la información de las características relevantes del tráfico de red.
2. **Organización de los registros:** Se seleccionan los registros correspondientes al tráfico IoT legítimo y al tráfico generado durante los ataques UDP Flood.
3. **Etiquetado de los datos:** Los registros se clasifican mediante las etiquetas Normal y UDP_Flood, según el escenario en el que fueron capturados.

Etapa 3: Preprocesamiento y selección de características

En esta etapa se preparan los datasets TON_IoT y experimental, para el entrenamiento de los modelos, manteniendo un tratamiento independendiente para los datasets y seleccionando características compatibles que permitan posteriormente comparar sus resultados mediante una información consolidada.

Actividades:

1. **Limpieza de datos:** Se identifican y tratan los valores nulos, infinitos, duplicados e inconsistencias en los registros y etiquetas.
2. **Homologación de características:** Se identifican las variables comunes o equivalentes entre ambos datasets para utilizar características comparables sin fusionar las fuentes de

datos, y se analiza la distribución de sus datos, para luego consolidarlas en una base de datos única.

3. **Transformación de variables:** Se codifican las variables categóricas y se escalan las variables numéricas según los requerimientos, para obtener variables ajustadas para usar en el aprendizaje automático.
4. **Selección de características:** Se analiza la correlación entre variables y se elimina características redundantes o que puedan generar fuga de información, para mantener un orden en la dataset consolidado.

Etapa 4: Entrenamiento y optimización de los modelos

En esta etapa se entrena y optimiza los modelos Random Forest y CNN utilizando de manera independiente el dataset TON_IoT y el dataset experimental, con el propósito de obtener modelos capaces de clasificar tráfico normal y ataques UDP Flood.

Actividades:

1. **División de los datos:** Cada dataset será dividido en conjuntos de entrenamiento, validación y prueba en proporciones de 70 %, 20 % y 10 %, respectivamente.
2. **Entrenamiento de los modelos:** Se entrenan los modelos Random Forest y CNN utilizando las características previamente seleccionadas.
3. **Optimización de los modelos:** Se ajustan los principales hiperparámetros de Random Forest y CNN y se aplicarán mecanismos de control del sobreajuste.
4. **Selección de las configuraciones óptimas:** Se seleccionan las configuraciones con mejor desempeño utilizando los resultados obtenidos en el conjunto de validación.

Etapa 5: Implementación, validación y visualización de resultados

En esta etapa se evalúa la capacidad de los modelos seleccionados para detectar ataques UDP Flood en nuevas capturas y presentar los resultados mediante un panel web para facilitar su análisis y comparación.

Actividades:

1. **Implementación del prototipo IDS:** Se desarrolla un prototipo de análisis offline que procesa nuevas capturas y clasifica los registros como tráfico Normal o UDP_Flood.

2. **Evaluación de los modelos:** Se calcula las métricas como Accuracy, Precision, Recall, F1-Score, FPR, FNR y latencia.
3. **Comparación de resultados:** Se compara el desempeño de Random Forest y CNN utilizando los resultados obtenidos con TON_IoT y el dataset experimental.

5. EXPERIMENTACIÓN

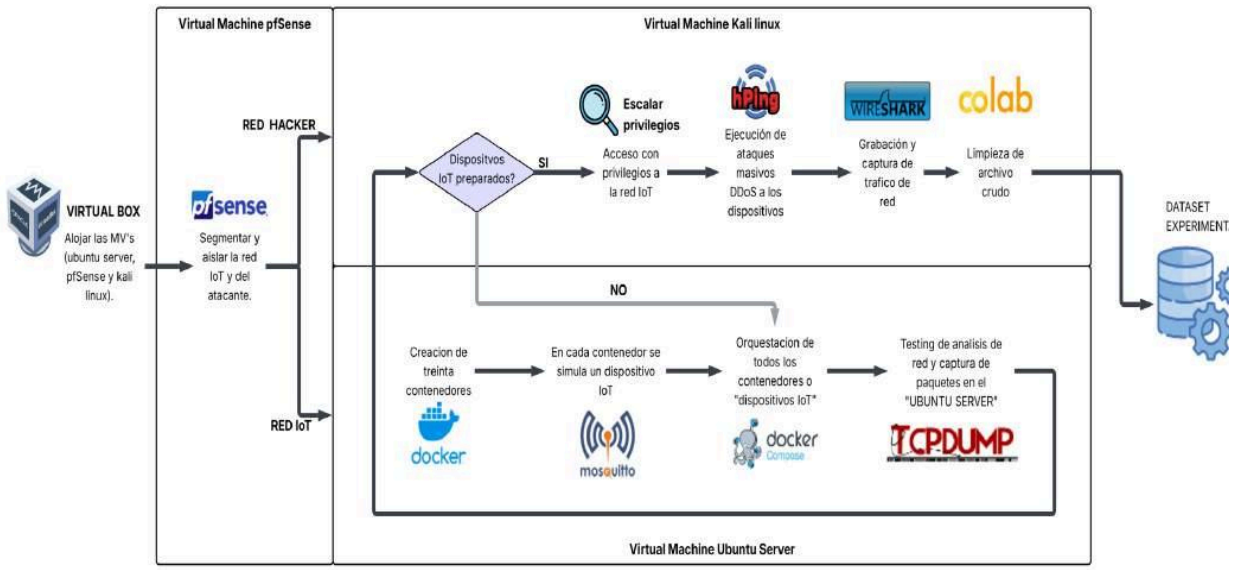
Etapa 1: Selección de datasets y diseño del entorno IoT

En esta etapa se selecciona las fuentes de datos y establecer un entorno IoT doméstico simulado que permita generar tráfico legítimo y ataques UDP Flood de manera controlada y aislada.

1. Selección del dataset TON_IoT: Se selecciona y analiza el dataset público TON_IoT, identificando las características y etiquetas relacionadas con tráfico de red y ataques. Además, presenta direcciones IP, duración de las comunicaciones, cantidad de bytes transmitidos y número de paquetes intercambiados
2. Configuración del entorno IoT simulado: El entorno experimental se implementó mediante VirtualBox, utilizando las máquinas virtuales Ubuntu Server, pfSense y Kali Linux. La infraestructura se configuró mediante una segmentación de red que separa la máquina atacante del entorno IoT, utilizando pfSense como elemento de enrutamiento y control entre ambos segmentos. Ubuntu Server funciona como servidor para la ejecución de los dispositivos IoT simulados mediante contenedores Docker, mientras que Kali Linux se utiliza para generar el tráfico correspondiente al escenario de ataque. Dentro de Ubuntu Server se implementó una red Docker de tipo bridge denominada red_iot, correspondiente al segmento 172.20.0.0/16, administrada mediante Docker Compose.

   Dentro de esta red se desplegaron contenedores destinados a representar diferentes dispositivos IoT domésticos. Se configuró un contenedor mosquitto_broker con dirección 172.20.0.2, encargado de proporcionar el servicio MQTT mediante el puerto TCP 1883, junto con los contenedores camara_1 (172.20.0.3), enchufe_1 (172.20.0.4) y foco_1 (172.20.0.5), que representan dispositivos IoT domésticos simulados. La comunicación entre estos componentes se realiza dentro de la red virtual de Docker. Asimismo, la interfaz de puente virtual asociada a la red de contenedores permite realizar la captura y supervisión del tráfico intercambiado entre los dispositivos y el broker.

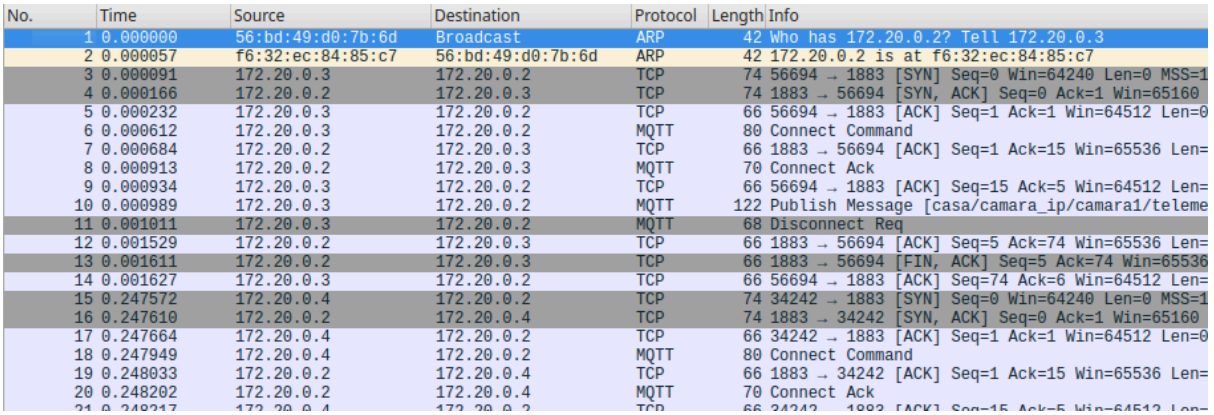
Figura 5.1
Entorno IoT Simulado



3. **Generación y captura de tráfico IoT legítimo:** Para representar el comportamiento normal de una red IoT doméstica, se generó tráfico legítimo mediante el protocolo MQTT. Los dispositivos IoT simulados ejecutan publicaciones periódicas hacia el broker Mosquitto. Durante esta actividad, el tráfico generado por los contenedores fue capturado mediante tcpdump y analizado posteriormente con Wireshark. La captura se realizó sobre la interfaz de puente de Docker durante un periodo controlado de 180 segundos, En la inspección del archivo PCAP se identificaron las principales fases de comunicación del tráfico legítimo, incluyendo la resolución ARP, el establecimiento de la conexión TCP mediante el three-way handshake, el establecimiento de la sesión MQTT, el intercambio de mensajes de publicación y el cierre de las conexiones. Estas capturas constituyen la fuente de los registros correspondientes al comportamiento normal del entorno experimental.

Figura 5.2

Captura de tráfico normal



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	56:bd:49:d0:7b:6d	Broadcast	ARP	42	Who has 172.20.0.2? Tell 172.20.0.3
2	0.000057	f6:32:ec:84:85:c7	56:bd:49:d0:7b:6d	ARP	42	172.20.0.2 is at f6:32:ec:84:85:c7
3	0.000091	172.20.0.3	172.20.0.2	TCP	74	56694 → 1883 [SYN] Seq=0 Win=64240 Len=0 MSS=1
4	0.000166	172.20.0.2	172.20.0.3	TCP	74	1883 → 56694 [SYN, ACK] Seq=0 Ack=1 Win=65160
5	0.000232	172.20.0.3	172.20.0.2	TCP	66	56694 → 1883 [ACK] Seq=1 Ack=1 Win=64512 Len=0
6	0.000612	172.20.0.3	172.20.0.2	MQTT	80	Connect Command
7	0.000684	172.20.0.2	172.20.0.3	TCP	66	1883 → 56694 [ACK] Seq=1 Ack=15 Win=65536 Len=
8	0.000913	172.20.0.2	172.20.0.3	MQTT	70	Connect Ack
9	0.000934	172.20.0.3	172.20.0.2	TCP	66	56694 → 1883 [ACK] Seq=15 Ack=5 Win=64512 Len=
10	0.000989	172.20.0.3	172.20.0.2	MQTT	122	Publish Message [casa/camara_ip/camara1/teleme
11	0.001011	172.20.0.3	172.20.0.2	MQTT	68	Disconnect Req
12	0.001529	172.20.0.2	172.20.0.3	TCP	66	1883 → 56694 [ACK] Seq=5 Ack=74 Win=65536 Len=
13	0.001611	172.20.0.2	172.20.0.3	TCP	66	1883 → 56694 [FIN, ACK] Seq=5 Ack=74 Win=65536
14	0.001627	172.20.0.3	172.20.0.2	TCP	66	56694 → 1883 [ACK] Seq=74 Ack=6 Win=64512 Len=
15	0.247572	172.20.0.4	172.20.0.2	TCP	74	34242 → 1883 [SYN] Seq=0 Win=64240 Len=0 MSS=1
16	0.247610	172.20.0.2	172.20.0.4	TCP	74	1883 → 34242 [SYN, ACK] Seq=0 Ack=1 Win=65160
17	0.247664	172.20.0.4	172.20.0.2	TCP	66	34242 → 1883 [ACK] Seq=1 Ack=1 Win=64512 Len=0
18	0.247949	172.20.0.4	172.20.0.2	MQTT	80	Connect Command
19	0.248033	172.20.0.2	172.20.0.4	TCP	66	1883 → 34242 [ACK] Seq=1 Ack=15 Win=65536 Len=
20	0.248202	172.20.0.2	172.20.0.4	MQTT	70	Connect Ack

4. **Generación y captura del ataque:** Se genera tráfico UDP Flood desde Kali Linux hacia Ubuntu Server y se capturará el tráfico mediante Wireshark y tcpdump para obtener los archivos PCAP.

Figura 5.3

Captura de tráfico UDP FLOOD

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.10.101	192.168.20.15	UDP	66	11917 → 1883 Len=0
2	0.000346	192.168.10.101	192.168.20.15	UDP	66	11918 → 1883 Len=0
3	0.000346	192.168.10.101	192.168.20.15	UDP	66	11919 → 1883 Len=0
4	0.000346	192.168.10.101	192.168.20.15	UDP	66	11920 → 1883 Len=0
5	0.000346	192.168.10.101	192.168.20.15	UDP	66	11921 → 1883 Len=0
6	0.000346	192.168.10.101	192.168.20.15	UDP	66	11922 → 1883 Len=0
7	0.000346	192.168.10.101	192.168.20.15	UDP	66	11923 → 1883 Len=0
8	0.000346	192.168.10.101	192.168.20.15	UDP	66	11924 → 1883 Len=0
9	0.000509	192.168.10.101	192.168.20.15	UDP	66	11925 → 1883 Len=0
10	0.000510	192.168.10.101	192.168.20.15	UDP	66	11926 → 1883 Len=0
11	0.000510	192.168.10.101	192.168.20.15	UDP	66	11927 → 1883 Len=0
12	0.000510	192.168.10.101	192.168.20.15	UDP	66	11928 → 1883 Len=0
13	0.000510	192.168.10.101	192.168.20.15	UDP	66	11929 → 1883 Len=0
14	0.000761	192.168.10.101	192.168.20.15	UDP	66	11974 → 1883 Len=0
15	0.000855	192.168.10.101	192.168.20.15	UDP	66	11975 → 1883 Len=0
16	0.000855	192.168.10.101	192.168.20.15	UDP	66	11976 → 1883 Len=0
17	0.000855	192.168.10.101	192.168.20.15	UDP	66	11977 → 1883 Len=0
18	0.000855	192.168.10.101	192.168.20.15	UDP	66	11978 → 1883 Len=0
19	0.000969	192.168.10.101	192.168.20.15	UDP	66	11979 → 1883 Len=0
20	0.000969	192.168.10.101	192.168.20.15	UDP	66	11980 → 1883 Len=0
21	0.000969	192.168.10.101	192.168.20.15	UDP	66	11981 → 1883 Len=0

Etapa 2: Construcción y etiquetado del dataset experimental

En esta etapa se construye el dataset experimental propio a partir de las capturas de tráfico obtenidas en el entorno IoT simulado, diferenciando el tráfico normal del tráfico correspondiente al ataque UDP Flood.

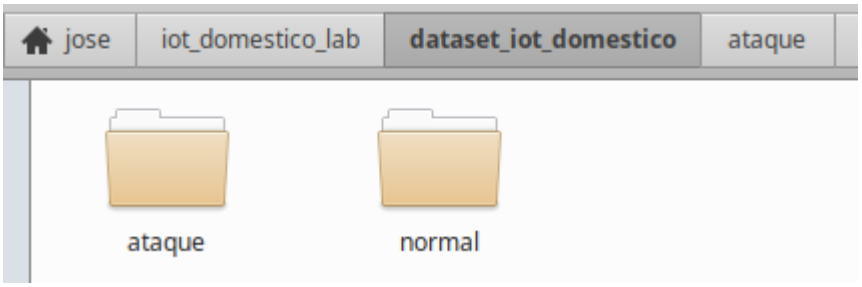
Actividades:

1. **Extracción de características:**Se desarrolló un script en Python denominado pcap_a_csv_tesis.py, utilizando la librería Scapy para inspeccionar y extraer información de los paquetes almacenados en los archivos PCAP. Se identificaron características como las direcciones IP de origen y destino, marcas de tiempo, puertos y tamaños de los paquetes. Posteriormente, mediante Pandas, se procesó esta información para calcular métricas de tráfico, como la duración de los flujos, la cantidad de bytes y el número de paquetes intercambiados, generando archivos CSV con ocho características para su utilización en los modelos de Machine Learning.

2. **Organización de los registros:**: Se organizan los archivos de captura PCAP obtenidos durante la experimentación en carpetas diferenciadas, separando aquellos correspondientes al tráfico IoT legítimo de los generados durante los ataques UDP Flood. Esta organización permite identificar el origen de cada captura y facilita su procesamiento posterior mediante el script desarrollado en Python, asegurando que los archivos sean convertidos a formato CSV con las etiquetas correspondientes a cada escenario experimental.

Figura 5.4

Carpetas de los registros correspondientes

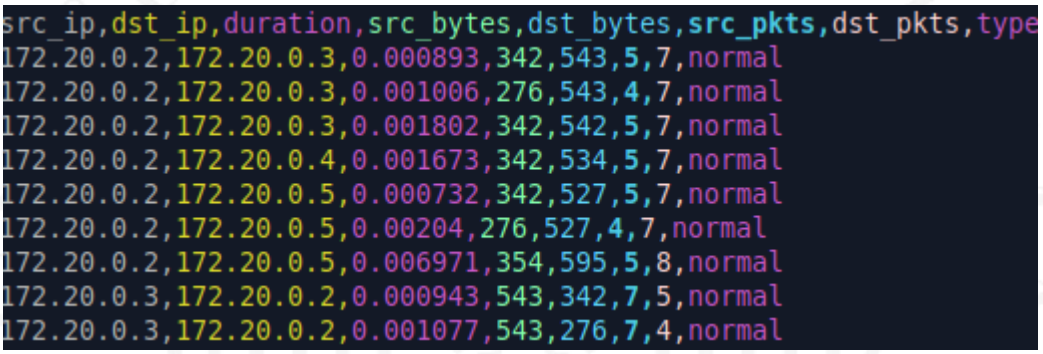


3. **Etiquetado de los datos:** Se asignan etiquetas a los registros según el escenario experimental del que proceden las capturas PCAP. Para ello, se ejecuta el script pcap_a_csv_scapy.py desde la terminal, indicando como argumentos el archivo de entrada, la ruta del archivo CSV de salida y la etiqueta correspondiente (normal o UDP_Flood). Este procedimiento se automatiza mediante un ciclo que recorre los archivos PCAP almacenados en sus respectivas carpetas, convirtiéndolos

individualmente a formato CSV. De esta manera, cada registro obtiene una etiqueta que permite diferenciar el tráfico legítimo del tráfico generado durante los ataques. Por ello.la variable src_ip representa la dirección IP de origen del flujo, mientras que dst_ip corresponde a la dirección IP de destino. duration indica la duración del flujo de comunicación, src_bytes y dst_bytes representan la cantidad de bytes transmitidos desde el origen hacia el destino y viceversa, respectivamente, y src_pkts y dst_pkts corresponden al número de paquetes enviados en cada dirección. Finalmente, type constituye la variable objetivo categórica, utilizada para identificar la clase de tráfico, diferenciando entre tráfico normal y tráfico asociado al ataque UDP Flood.

Figura 5.5

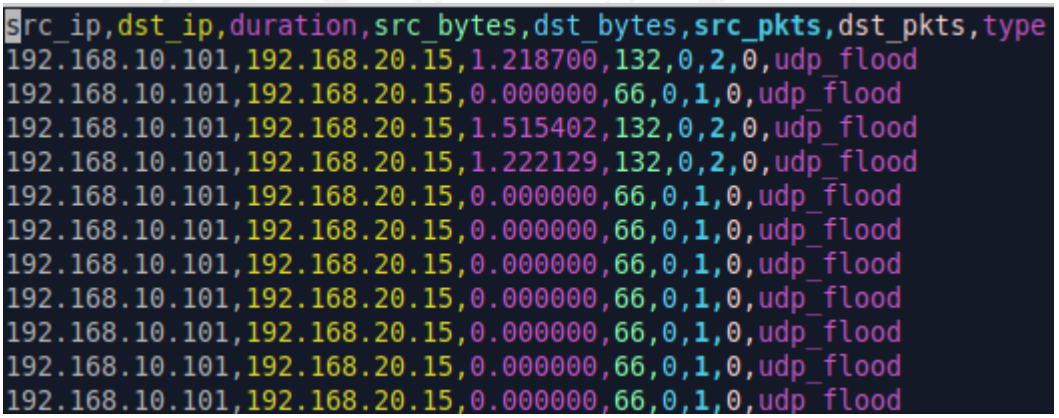
Contenido del tráfico normal en un archivo csv etiquetado



```
src_ip,dst_ip,duration,src_bytes,dst_bytes,src_pkts,dst_pkts,type
172.20.0.2,172.20.0.3,0.000893,342,543,5,7,normal
172.20.0.2,172.20.0.3,0.001006,276,543,4,7,normal
172.20.0.2,172.20.0.3,0.001802,342,542,5,7,normal
172.20.0.2,172.20.0.4,0.001673,342,534,5,7,normal
172.20.0.2,172.20.0.5,0.000732,342,527,5,7,normal
172.20.0.2,172.20.0.5,0.00204,276,527,4,7,normal
172.20.0.2,172.20.0.5,0.006971,354,595,5,8,normal
172.20.0.3,172.20.0.2,0.000943,543,342,7,5,normal
172.20.0.3,172.20.0.2,0.001077,543,276,7,4,normal
```

Figura 5.6

Contenido del tráfico UDP Flood en un archivo csv etiquetado



```
src_ip,dst_ip,duration,src_bytes,dst_bytes,src_pkts,dst_pkts,type
192.168.10.101,192.168.20.15,1.218700,132,0,2,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,1.515402,132,0,2,0,udp_flood
192.168.10.101,192.168.20.15,1.222129,132,0,2,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
192.168.10.101,192.168.20.15,0.000000,66,0,1,0,udp_flood
```

4.

Etapa 3: Preprocesamiento y selección de características

En esta etapa se preparan los datasets TON_IoT y experimental para el entrenamiento de los modelos, manteniéndolos independientes y seleccionando características compatibles que permitan posteriormente comparar sus resultados.

Actividades:

1. **Limpieza de datos:** El proceso de limpieza de datos consiste en identificar y tratar las anomalías presentes en los datasets TON_IoT y experimental, con el propósito de garantizar la calidad de los registros antes del entrenamiento de los modelos. En primer lugar, se verifica la presencia de valores nulos (NaN) e infinitos (∞), que pueden aparecer durante el procesamiento de las características o el cálculo de las métricas de flujo. Asimismo, se identifican y eliminan los registros duplicados para evitar la repetición innecesaria de observaciones que puedan afectar la distribución de los datos. Finalmente, se revisan las inconsistencias en los tipos de datos, realizando las conversiones necesarias de variables categóricas o textuales a formatos numéricos cuando corresponda. De esta manera, se obtienen registros consistentes y adecuados para las siguientes etapas del procesamiento.
2. **Homologación de características:** La homologación de características permite establecer una estructura de atributos comparables entre los datasets TON_IoT y experimental, manteniendo ambas fuentes de información independientes. Para ello, se identifican las variables comunes o equivalentes relacionadas con el comportamiento del tráfico de red, considerando principalmente las direcciones IP de origen y destino, la duración de los flujos, la cantidad de bytes transmitidos y el número de paquetes intercambiados. Posteriormente, se revisan los tipos de datos y la correspondencia entre las variables para establecer una estructura compatible que permita realizar los experimentos bajo criterios similares. Asimismo, se unifican las etiquetas de clasificación mediante un esquema binario, donde el valor 0 representa el tráfico normal y el valor 1 corresponde al ataque UDP Flood. Este procedimiento permite mantener una estructura de entrada comparable sin combinar los registros de ambos datasets.

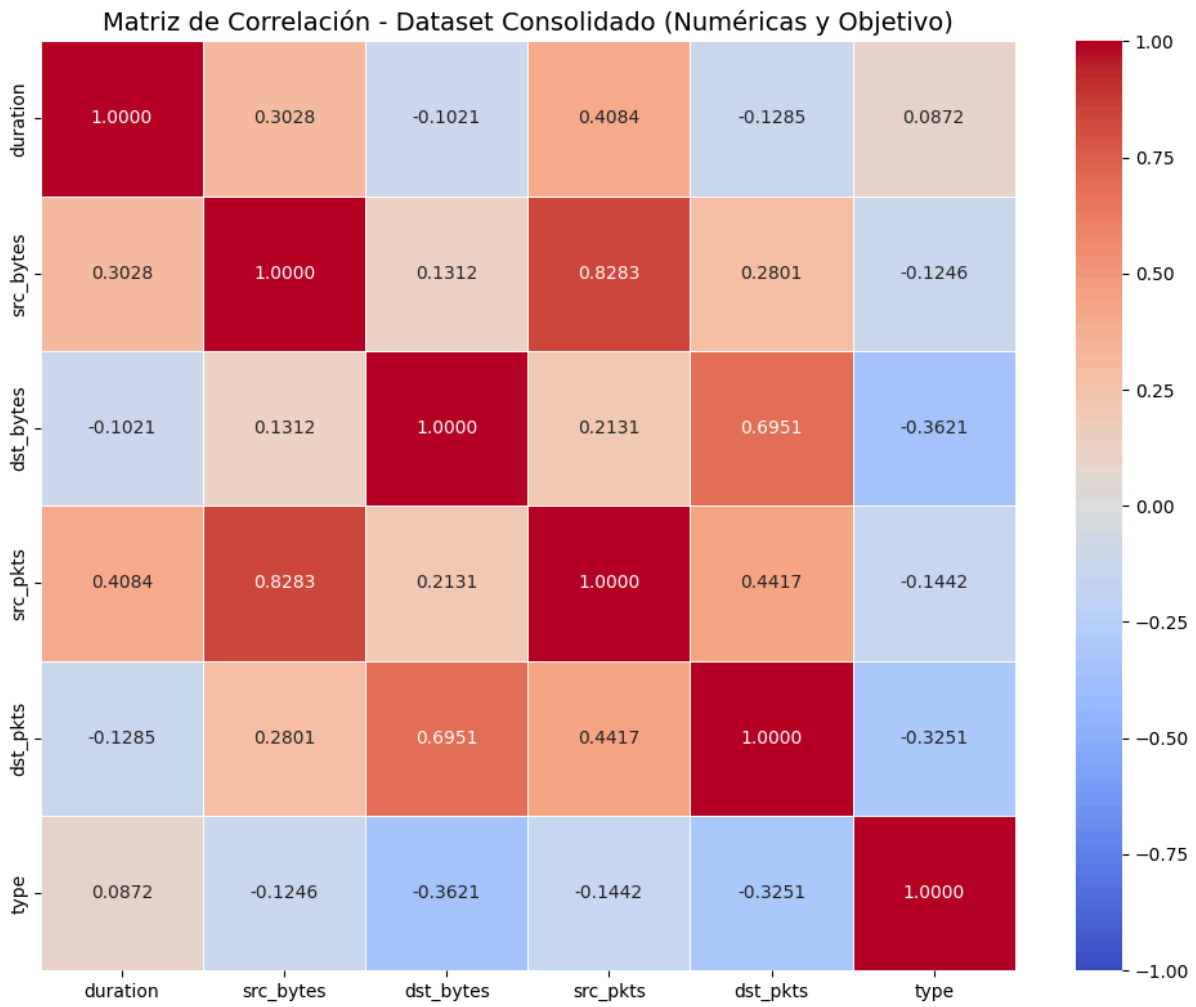
3. **Transformación de variables:** La transformación de variables permite adecuar los datos a los formatos requeridos por los algoritmos de Machine Learning y Deep Learning

seleccionados. En primer lugar, se realiza la codificación de las variables categóricas para convertirlas en representaciones numéricas que puedan ser procesadas por los modelos. Posteriormente, se aplica el escalamiento de las características numéricas, como la duración de los flujos, la cantidad de bytes transmitidos y el número de paquetes, especialmente para el modelo CNN, con el propósito de mantener sus valores dentro de rangos comparables y facilitar el proceso de aprendizaje. Adicionalmente, se analiza la distribución de las clases de tráfico para identificar posibles desequilibrios entre los registros normales y los correspondientes al ataque UDP Flood. En caso de ser necesario, se aplican técnicas de balanceo sobre el conjunto de entrenamiento para evitar que el modelo favorezca a la clase mayoritaria.

4. **Selección de características:** Se identifican los atributos más relevantes para clasificar el tráfico de red, mediante matrices de correlación y mapas de calor (heatmaps), con el fin de detectar variables redundantes y reducir el sobreajuste (overfitting). Se observa una correlación positiva entre src_bytes y src_pkts (r = 0.8283), así como entre dst_bytes y dst_pkts (r = 0.6951), lo que indica que una mayor cantidad de paquetes se relaciona con un mayor volumen de datos transmitidos. Asimismo, se identifica una correlación negativa entre la etiqueta del ataque (type) y las variables dst_bytes (r = -0.3621) y dst_pkts (r = -0.3251), lo que evidencia una menor cantidad de respuestas del destino durante el UDP Flood. Finalmente, se evalúan las direcciones IP (src_ip y dst_ip) y se seleccionan las características de entrada para los modelos,evitando utilizar información del conjunto de prueba para prevenir la fuga de información (data leakage) y favorecer la generalización de los modelos.

Figura 5.7

Matriz de correlación



Etapa 4: Entrenamiento y optimización de los modelos

En esta etapa se entrena y optimiza los modelos Random Forest y CNN utilizando de manera independiente el dataset TON_IoT y el dataset experimental, con el propósito de obtener modelos capaces de clasificar tráfico normal y ataques UDP Flood.

Actividades:

1. **División de los datos:**Cada dataset se divide en tres conjuntos: entrenamiento (70 %), validación (20 %) y prueba (10 %). El conjunto de entrenamiento se utiliza para que los modelos aprendan los patrones y características asociados al tráfico normal y a los ataques UDP Flood. Por su parte, el conjunto de validación permite evaluar el comportamiento de los modelos durante el entrenamiento y orientar los ajustes de sus

parámetros. Finalmente, el conjunto de prueba se reserva para realizar una evaluación independiente del desempeño de las configuraciones seleccionadas.

2. **Entrenamiento de los modelos:**En esta actividad se entrenan de manera independiente los modelos Random Forest, XGBoost, CNN 1D, CNN 1D + Random Forest y CNN 1D + Bi-LSTM utilizando el conjunto de entrenamiento de TON_IoT y del dataset experimental. Los algoritmos procesan los registros de tráfico normal y ataques UDP Flood mediante distintas metodologías: Random Forest y XGBoost construyen árboles de decisión sobre los datos tabulares; CNN 1D aprende representaciones no lineales; el modelo híbrido (CNN 1D + RF) combina la extracción convolucional de características con el clasificador basado en árboles; y CNN 1D + Bi-LSTM añade el análisis de dependencias temporales y secuenciales. Con ello se obtienen las distintas arquitecturas entrenadas para su posterior optimización y evaluación.
3. **Optimización de los modelos:** En esta actividad se ajustan los principales hiperparámetros d elos modelos con el propósito de mejorar su capacidad de clasificación y obtener un desempeño adecuado durante la detección de ataques UDP Flood. Para Random Forest se consideran parámetros relacionados con la cantidad de árboles, la profundidad máxima y los criterios de división. En el caso de CNN, se ajustan aspectos como el número de capas, filtros, tamaño del lote (batch size) y número de épocas (epochs). Asimismo, se consideran mecanismos de control del sobreajuste para evitar que los modelos se adapten excesivamente a los datos de entrenamiento y presenten dificultades al clasificar registros nuevos. Los ajustes se realizan utilizando los resultados obtenidos en el conjunto de validación.
4. **Selección de las configuraciones óptimas:**En esta actividad se analizan los resultados obtenidos durante el entrenamiento y la optimización para seleccionar las configuraciones con mejor desempeño de los modelos. Para ello, se comparan los resultados de las diferentes configuraciones utilizando las métricas obtenidas en el conjunto de validación, considerando principalmente Accuracy, Precision, Recall y F1-Score. Asimismo, se revisa el comportamiento de los modelos para identificar posibles diferencias en su capacidad de reconocer el tráfico normal y los ataques UDP Flood. Finalmente, se seleccionan las configuraciones que presentan el mejor equilibrio entre capacidad de clasificación y control del sobreajuste, cuyos modelos serán utilizados posteriormente en el prototipo IDS.

Etapa 5: Implementación, validación y visualización de resultados

En esta etapa se evalúa la capacidad de los modelos seleccionados para detectar ataques UDP Flood en nuevas capturas y presentar los resultados mediante un panel web para facilitar su análisis y comparación.

Actividades:

1. **Implementación del prototipo IDS:** En esta actividad se desarrolla un prototipo IDS de análisis offline mediante Python, integrando los modelos previamente entrenados y seleccionados. El prototipo recibe archivos de captura de tráfico de red en formato PCAP, a partir de los cuales se extraen las características necesarias para realizar la clasificación. Posteriormente, los registros obtenidos pasan por el mismo proceso de transformación y preparación utilizado durante el entrenamiento, garantizando la compatibilidad con los modelos. Finalmente, ambos algoritmos procesan los registros de manera independiente y generan predicciones que permiten identificar el tráfico como Normal o UDP_Flood. De esta manera, se obtiene una herramienta experimental que permite analizar nuevas capturas del entorno IoT simulado. d.
2. **Evaluación de los modelos:** En esta actividad se evalúa el desempeño de los modelos utilizando los conjuntos de prueba de TON_IoT y del dataset experimental. Para ello, se calculan métricas como Accuracy, Precision, Recall y F1-Score, que permiten medir la capacidad de clasificación y detección de los ataques UDP Flood. Asimismo, se analizan la tasa de falsos positivos (FPR) y la tasa de falsos negativos (FNR), con el propósito de identificar los errores cometidos durante la clasificación. Adicionalmente, se mide la latencia o tiempo de respuesta de los modelos para determinar cuánto tiempo requieren para procesar los registros y generar sus predicciones. Los resultados permiten evaluar tanto la precisión de detección como la eficiencia de cada algoritmo.
3. **Comparación de resultados:** En esta actividad se comparan los resultados obtenidos por los modelos a partir de los experimentos realizados con TON_IoT y el dataset experimental. Para ello, se organizan las métricas de evaluación de cada modelo en tablas comparativas, considerando Accuracy, Precision, Recall, F1-Score, FPR, FNR y latencia. Asimismo, se analizan las diferencias en el comportamiento de los algoritmos, identificando cuál presenta mejores resultados en la clasificación del tráfico normal y la detección de ataques UDP Flood. Esta comparación se realiza por separado para cada dataset, con el propósito de reconocer las fortalezas y limitaciones de ambos modelos en los distintos escenarios de evaluación.

6. RESULTADOS

En cuanto a los estudios revisados, se observa que los algoritmos de Machine Learning y Deep Learning han obtenido buenos resultados en la detección de ataques DDoS en entornos IoT. Para Pandey y Mishra (2023) y Ebady Manaa et al. (2024) concluyeron que el algoritmo random forest presenta un buen rendimiento y una disminución de tiempos de procesamiento en la identificación de tráfico malicioso, aunque realizaron estudio comparativos con algoritmos de deep learning, estas no mostraron un exactitud superior al RF. Por otro lado, Junior et al. (2025) demostró que la combinación de CNN y Bi-LSTM permite detectar diferentes patrones de ataques, alcanzando una exactitud superior al 99 % en los conjuntos de datos evaluados. Asimismo, Abulhassan et al. (2025) señalaron que los modelos de Deep Learning pueden lograr una alta capacidad de detección, aunque algunos requieren mayores recursos computacionales en comparación con los algoritmos tradicionales. En general, estos estudios muestran que existen diferentes alternativas para detectar ataques DDoS, pero se debe resaltar la importancia de comparar su rendimiento y consumo de recursos. Esto resulta importante para la investigación, ya que se identifican ataques UDP Flood u otros ataques, a dispositivos IoT domésticos mediante modelos individuales e híbridos de Machine Learning y Deep Learning.

Tabla 2.

Tabla de métricas para los ataques DDoS.

	Accuracy	**Precision**	**Recall**	**F1-Score**	**Latency**
Random Forest	0.990602	0.996347	0.985992	0.991142	0.011607
XGBoost	0.989639	0.99408	0.986444	0.990247	0.0029
CNN 1D	0.981446	0.981515	0.983732	0.982622	0.068252
CNN 1D - Random Forest	0.991325	0.996805	0.986896	0.991826	0.079192
CNN 1D - Bi-LSTM	0.980964	0.981064	0.983281	0.982171	0.098763

Se observa que en situaciones de ataque UDP Flood, los de mejor accuracy y precisión, son el random forest y el modelo híbrido CNN 1D - random forest. Y en términos de mejor latencia se lleva el XGBoost. Cabe mencionar que es obvio que entre todos los modelos, el XGBoost debería mostrar mucha menor latencia, por su adaptabilidad a encontrar características importantes y su control de sobreajuste. Para investigación, se puede optar por usar dos modelos mas capaces en este contexto de la investigación, el modelo híbrido de CNN 1D con Random Forest logra la mayor exactitud en la detección de ataques UDP Flood en dispositivos domésticos, superando ligeramente a los demás modelos, además esto se respalda que los modelos de machine learning demostraron mejor desempeño en otras métricas. En cuanto a la velocidad de respuesta, los algoritmos tradicionales como XGBoost demuestra ser mucho más rápidos, mientras que los modelos que incluyen Deep Learning tardaron un poco más en procesar la información. Esto demuestra que, aunque combinar técnicas ayuda a mejorar el acierto frente a las amenazas, se debe equilibrar la ganancia en precisión con el tiempo que tarda el sistema en reaccionar para proteger los equipos del hogar en tiempo real.

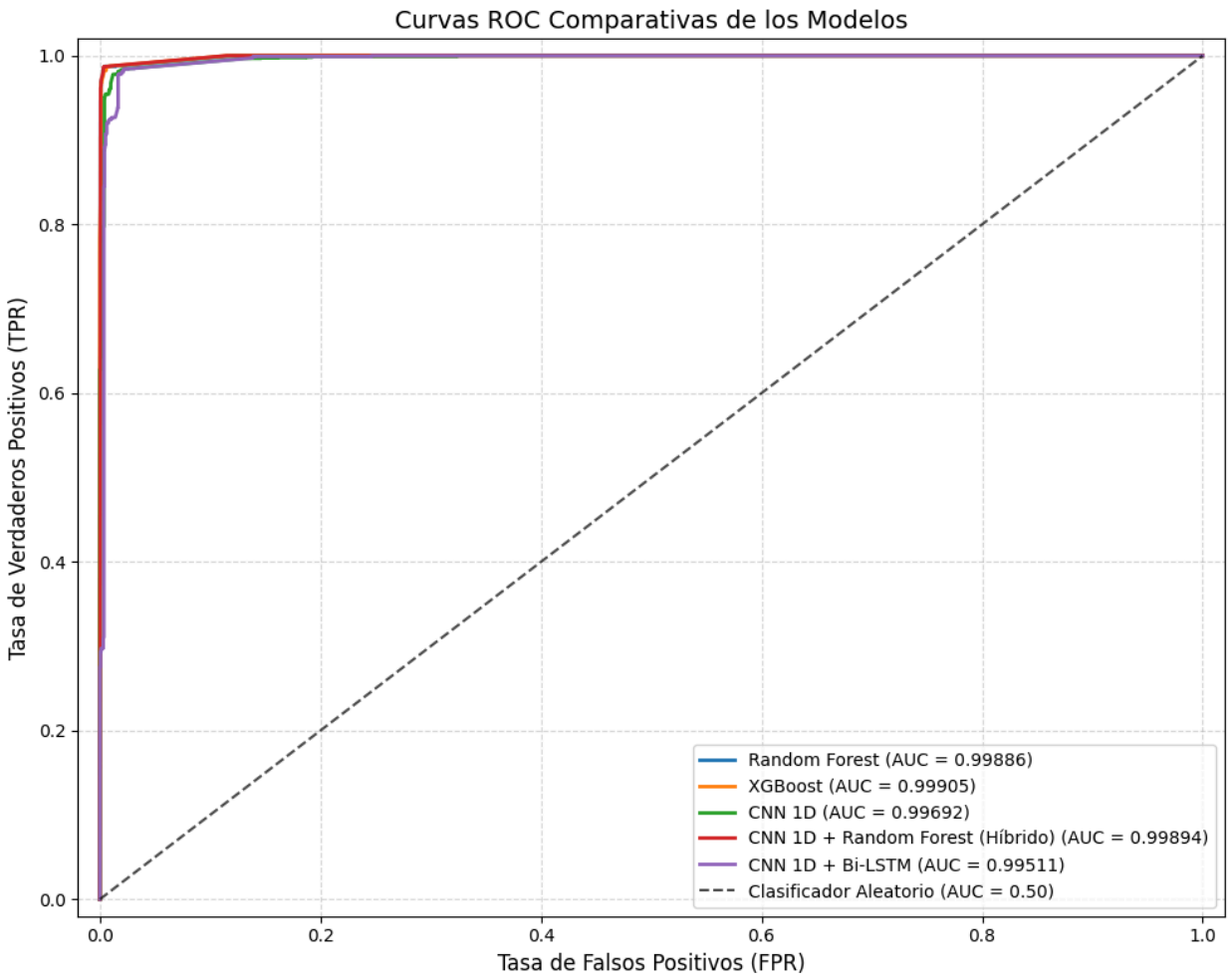


Figura 3. Gráfica ROC de los modelos.

Cabe mencionar que en la evaluación del modelo, los modelos mas calificados para esta investigación son el modelo de random forest y el modelo híbrido CNN 1D + random forest. Por lo tanto, se concluye que modelos vinculados con ML es el mas adecuado para esta investigación, considerando que los dispositivos son domésticos. Mientras que los modelos de Deep Learning y las arquitecturas híbridas, como CNN + Bi-LSTM, muestran una elevada capacidad para reconocer patrones complejos de tráfico malicioso.

DECLARACIONES

Uso de inteligencia artificial generativa

"Durante la elaboración de este artículo se utilizó ChatGPT-6 como herramienta de apoyo para la revisión de la redacción, mejora de la claridad y organización del contenido académico"

REFERENCIAS

Abualhassan, A., Rashid, I., Binbeshr, F., & Imam, M. (2025). DDoS attack detection in IoT: A comparative resource and performance analysis of deep learning and machine learning models. IEEE Access, 13, 116529–116547. https://doi.org/10.1109/ACCESS.2025.3583855

Assy, A. T., Mostafa, Y., Abd El-khaleq, A., & Mashaly, M. (2023). Anomaly-based intrusion detection system using one-dimensional convolutional neural network. Procedia Computer Science, 220, 78–85. https://doi.org/10.1016/j.procs.2023.03.013

Bankó, M. B., Dyszewski, S., Králová, M., Dragoni, N., & others. (2025). Advancements in machine learning-based intrusion detection in IoT: Research trends and challenges. Algorithms, 18(4), 209. https://doi.org/10.3390/a18040209

Chen, S., Yu, J., & Wang, S. (2022). One-dimensional convolutional neural network-based active feature extraction for fault detection and diagnosis of industrial processes and its understanding via visualization. ISA Transactions, 122, 424–443. https://doi.org/10.1016/j.isatra.2021.04.042

Alsubaei, F. S. (2025). Smart deep learning model for enhanced IoT intrusion detection. Scientific Reports, 15, 20577. https://doi.org/10.1038/s41598-025-06363-5

Amine, M. S., Nada, F. A., & Hosny, K. M. (2025). Improved model for intrusion detection in the Internet of Things. Scientific Reports, 15, 21547. https://doi.org/10.1038/s41598-025-92852-6

Ebrahimi, F., et al. (2025). Intrusion detection in IoT using convolutional neural networks: An explainable AI approach. Cybersecurity, 8, 66. https://doi.org/10.1186/s42400-025-00369-2

Pandey, V. K., et al. (2025). A lightweight framework to secure IoT devices with limited resources in cloud environments. Scientific Reports, 15, 26009. https://doi.org/10.1038/s41598-025-09885-0

Praveen, S. J., Prahladh, V., & Devi, B. S. K. (2025). Host-based intrusion detection framework for IoT. ICIMA 2025. https://doi.org/10.1109/ICIMA64861.2025.11074165

Yang, K., et al. (2025). NIDS-CNNRF: Integrating CNN and Random Forest for intrusion detection. Future Internet. https://doi.org/10.1016/j.iot.2025.101607

Reed, A., Dooley, L., & Mostefaoui, S. K. (2025). SA-IDS: Slow DoS detection in IoT networks. Internet of Things, 30, 101512. https://doi.org/10.1016/j.iot.2025.101512

Junior, A. J. K. Q., Tchao, E. T., Keelson, E., Agbemenu, A. S., Nunoo-Mensah, H., & Yeboah-Akowuah, B. (2025). CDBi-LSTM: A hybrid deep learning model with

attention-based fusion for efficient DDoS detection in IoT environments. IET Communications, 19(1), e70094. https://doi.org/10.1049/cmu2.70094

Tian, J., et al. (2025). Evaluating AI-driven intrusion detection for IoT. PeerJ Computer Science, 11, e3352. https://doi.org/10.7717/peerj-cs.3352

Ganji, K., Afshan N. (2025), "A bibliometric review of Internet of Things (IoT) on cybersecurity issues". Journal of Science and Technology Policy Management, Vol. 16 No. 6 pp. 984–1002. https://doi.org/10.1108/JSTPM-05-2023-0071

Ko, H., Ogiela, M., et al. (2025). Optimizing Network Intrusion Detection Performance with GNN-Based Feature Selection. Computers, Materials & Continua. https://doi.org/10.32604/cmc.2025.065885

T, S. & Mamatha, V. (2025). AI-Based Intrusion Detection in IoT Networks Using Lightweight Deep Learning Models. International Journal of Emerging Research in Science, Engineering, and Management, 1(4), 1–8. https://doi.org/10.58482/ijersem.v1i4.1

Wahab, S., Sultana, S,, et al. (2025). A Multi-Class Intrusion Detection System for DDoS Attacks in IoT Networks Using Deep Learning and Transformers. Sensors, 25(15), 4845. https://doi.org/10.3390/s25154845

Adamova, A., Zhukabayeva, T., & Adamov, N. (2024). Machine learning algorithms for intrusion detection in IoT-enabled smart homes. Procedia Computer Science, 241, 427–432. https://doi.org/10.1016/j.procs.2024.08.059

Ahmed, A., Shah, A., Abdullah, A., Laghari, S. U. A., & Alwan, A. (2024). Detection of DDoS attacks in IoT networks using machine learning algorithms. 2024 International Conference on Engineering and Emerging Technologies (ICETAS). https://doi.org/10.1109/ICETAS62372.2024.11120083

Al Farsi, A., Khan, A., & Bait-Suwailam, M. M. (2024). A practical evaluation of intrusion detection in IoT networks using Random Forest and NIDS dataset. IEEE International Conference on Computing and Data Analytics (ICCDA). https://doi.org/10.1109/ICCDA64887.2024.10867402

Alabdulatif, A., et al. (2024). Machine learning enabled real-time IoT targeted DoS/DDoS attack detection. Journal of Network and Systems Management. https://doi.org/10.32604/cmc.2024.054610

Prümmer, J., van Steen, T., & van den Berg, B. (2024). A systematic review of cybersecurity training methods. Computers & Security, 136, 103585. https://doi.org/10.1016/j.cose.2023.103585

Rahman, M., et al. (2024). A survey on intrusion detection system in IoT networks. Cyber Security and Applications. https://doi.org/10.1016/j.csa.2024.100082

Srivastava, A., Tiwari, S., Rawat, B. S., & Dhondiyal, S. A. (2024). DDoS attacks detection in IoT using Naive Bayes and Random Forest. 2024 International Conference on Advanced Computing and Intelligent Engineering (ICAAIC). https://doi.org/10.1109/ICAAIC60222.2024.10574903

Talukder, M. A.,et al. (2024). Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. Journal of Big Data, 11, Article 33. https://doi.org/10.1186/s40537-024-00886-w

Garba, U. H., Toosi, A. N., Pasha, M. F., & Khan, S. (2024). SDN-based detection and mitigation of DDoS attacks on smart homes. Computer Communications, 221, 29–41. https://doi.org/10.1016/j.comcom.2024.04.001

Pakmehr, A., Aßmuth, A., Taheri, N., et al. (2024). DDoS attack detection techniques in IoT networks: A survey. Cluster Computing, 27, 14637–14668. https://doi.org/10.1007/s10586-024-04662-6

Hizal, S., Cavosuglu, U., et al. (2024), A novel deep learning-based intrusion detection system for IoT DDoS security. Internet of Things, 28, 101336. https://doi.org/10.1016/j.iot.2024.101336

Alqarni, T. M., Hamadneh, B. M., & Jdaitawi, M. T. (2024). Perceived usefulness of IoT in the quality of life of special needs and elderly individuals. Heliyon, 10(3), e25122. https://doi.org/10.1016/j.heliyon.2024.e25122

Ebady Manaa, M., Hussain, S. M., Alasadi, S. A., & Al-Khamees, H. A. A. (2024). DDoS attacks detection based on machine learning algorithms in IoT environments. Inteligencia Artificial, 27(74), 152–165. https://doi.org/10.4114/intartif.vol27iss74pp152-165

Pazos, F. (2024). Performance evaluation of MQTT broker servers deployed in the cloud. Electronic Journal of SADIO, 23(1), 117–132. https://doi.org/10.24215/15146774e043

Rami, R. (2024). An ensemble boosting algorithm-based IDS for smart IoT. Journal of Intelligent Systems and IoT, 13(2), 272–292. https://doi.org/10.54216/jisiot.130222

Alzu, L., Albachayreh, A., et la. (2024)Explainable AI-Based DDoS Attacks Classification Using Deep Transfer Learning. Computers, Materials & Continua. https://doi.org/10.32604/cmc.2024.052599

Kikissagbe, B. R., & Adda, M. (2024). Machine learning-based intrusion detection methods in IoT systems: A comprehensive review. Electronics, 13(18), 3601. https://doi.org/10.3390/electronics13183601

Shakya, V., & Abbas, R. (2024). A Comparative Analysis of Machine Learning Models for DDoS Detection in IoT Networks. arXiv preprint arXiv:2411.05890. https://arxiv.org/abs/2411.05890

Sheeraz, M., et al. (2024). Revolutionizing SIEM security: An innovative correlation engine. Sensors, 24(15), 4901. https://doi.org/10.3390/s24154901

Bhavsar, M., Roy, K., Kelly, J., & Olusola, O. (2023). Anomaly-based intrusion detection system for IoT application. Discover Internet of Things, 3, Article 5. https://doi.org/10.1007/s43926-023-00034-5

Jemal, A., et al. (2023). IoT DoS and DDoS attack detection using CNN. IEEE Conference Proceeding. https://ieeexplore.ieee.org/document/10337567

Thereza, N., & Ramli, K. (2023). Development of intrusion detection models for IoT utilizing CICIoT2023 dataset. 2023 IEEE International Conference on Smart Internet of Things (ICON-SONICS). https://doi.org/10.1109/ICON-SONICS59898.2023.10435006

Pandey, N., & Mishra, P. K. (2023). Detection of DDoS attack in IoT traffic using ensemble machine learning techniques. Networks and Heterogeneous Media, 18(4), 1393–1409. https://doi.org/10.3934/nhm.2023061

Dutta, M., Malik, L., et la. (2023). Feature Engineering and Machine Learning Framework for DDoS Attack Detection in the Standardized Internet of Things. IEEE Internet of Things Journal, 10(10), 8658–8669. https://doi.org/10.1109/JIOT.2023.3245153

N., M. & G., E. (2023). Protecting IoT Servers Against Flood Attacks with the Quasi Deterministic Transmission Policy. IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). https://doi.org/10.1109/TrustCom60117.2023.00068

Glas, M., Vielberth, M., & Pernul, G. (2023). Train as you fight: Evaluating authentic cybersecurity training in cyber ranges. CHI Conference on Human Factors in Computing Systems. https://doi.org/10.1145/3544548.3581046

Wang, Z., Du, R., et la. (2023). IoT Intrusion Detection Model based on CNN-GRU. Frontiers in Computing and Intelligent Systems, 4(2). https://doi.org/10.54097/fcis.v4i2.10302

Parsons, E. K., Panaousis, E., Loukas, G., & Sakellari, G. (2023). A Survey on Cyber Risk Management for the Internet of Things. Preprints. https://doi.org/10.20944/preprints202306.2172.v1

Abdelhady, G., Hussein, K. W., & Gad, I. A. A. (2026). Privacy-preserving intrusion detection in IoT smart homes using a federated hybrid 1D-CNN–LSTM model with explainable AI. *Scientific Reports, 16*, 27559. https://doi.org/10.1038/s41598-026-67450-9

Javed, A., Ehtsham, A., Jawad, M., Awais, M. N., Qureshi, A.-U.-H., & Larijani, H. (2024). Implementation of lightweight machine learning-based intrusion detection system on IoT devices of smart homes. *Future Internet, 16*(6), 200. https://doi.org/10.3390/fi16060200

Chintapalli, S. S. N., Singh, S. P., Frnda, J., Divakarachari, P. B., Sarraju, V. L., & Falkowski-Gilski, P. (2024). OOA-modified Bi-LSTM network: An effective intrusion detection framework for IoT systems. *Heliyon, 10*(8), e29410. https://doi.org/10.1016/j.heliyon.2024.e29410

Omarov, V., S., Z., B., A., et la. (2023). One Dimensional Conv-BiLSTM Network with Attention Mechanism for IoT Intrusion Detection. *Computers, Materials & Continua, 77*(3), 3765–3781. https://doi.org/10.32604/cmc.2023.042469

Hnamte, V., & Hussain, J. (2023). DCNNBiLSTM: An Efficient Hybrid Deep Learning-Based Intrusion Detection System. *Telematics and Informatics Reports, 10*, 100053. https://doi.org/10.1016/j.teler.2023.100053

Zhang, P., Jia, Y., & Shang, Y. (2022). Research and application of XGBoost in imbalanced data. International Journal of Distributed Sensor Networks, 18(6). https://doi.org/10.1177/15501329221106935